Lista Dwukierunkowa

Wygenerowano za pomocą Doxygen 1.17.0

1 Indeks klas 1

1.1 Lista klas . . . 1

2 Indeks plików 3

2.1 Lista plików . . . 3

3 Dokumentacja klas 5

3.1 Dokumentacja klasy DoublyLinkedList . . . 5

3.1.1 Dokumentacja konstruktora i destruktora . . . 6

3.1.1.1 ~DoublyLinkedList() . . . 6

3.1.2 Dokumentacja funkcji składowych . . . 6

3.1.2.1 clear() . . . 6

3.1.2.2 pop_back() . . . 6

3.1.2.3 pop_front() . . . 6

3.1.2.4 push_back() . . . 6

3.1.2.5 push_front() . . . 7

3.2 Dokumentacja klasy ListIterator . . . 7

3.2.1 Dokumentacja funkcji składowych . . . 7

3.2.1.1 operator"!=() . . . 7

3.2.1.2 operator++() . . . 8

3.2.1.3 operator--() . . . 8

3.2.1.4 operator==() . . . 8

3.3 Dokumentacja klasy Node . . . 8

3.3.1 Opis szczegółowy . . . 9

3.3.2 Dokumentacja konstruktora i destruktora . . . 9

3.3.2.1 Node() . . . 9

3.4 Dokumentacja klasy NodeFactory . . . 9

3.4.1 Opis szczegółowy . . . 9

3.4.2 Dokumentacja funkcji składowych . . . 10

3.4.2.1 createNode() . . . 10

4 Dokumentacja plików 11

4.1 DoublyLinkedList.h . . . 11

4.2 Iterator.h . . . 11

4.3 Dokumentacja pliku node.h . . . 12

4.3.1 Opis szczegółowy . . . 12

4.4 node.h . . . 12

4.5 Dokumentacja pliku NodeFactory.h . . . 12

4.5.1 Opis szczegółowy . . . 12

4.6 NodeFactory.h . . . 13

Rozdział 1

Indeks klas

1.1 Lista klas

Tutaj znajdują się klasy, struktury, unie i interfejsy wraz z ich krótkimi opisami:

DoublyLinkedList 5
ListIterator 7
Node
    Klasa przechowująca pojedynczy element listy dwukierunkowej 8
NodeFactory
    Fabryka tworząca nowe obiekty węzłów listy 9

Rozdział 2

Indeks plików

2.1 Lista plików

Tutaj znajduje się lista wszystkich udokumentowanych plików wraz z ich krótkimi opisami:

DoublyLinkedList.h 11
Iterator.h 11
node.h
Definicja klasy Node reprezentującej pojedynczy węzeł listy 12
NodeFactory.h
Definicja klasy NodeFactory odpowiedzialnej za wzorzec fabryki węzłów 12

Rozdział 3

Dokumentacja klas

3.1 Dokumentacja klasy DoublyLinkedList

Metody publiczne

- **DoublyLinkedList** ()

  Rozmiar listy do obsługi indeksów.
- ~DoublyLinkedList ()
- void push_front (int val)

  Destruktor - czyści pamięć po usunięciu listy.
- void push_back (int val)

  Dodaje element na początek.
- void **insert_at** (int index, int val)

  Dodaje element na koniec.
- void pop_front ()

  Dodaje element z podanego indeksu.
- void pop_back ()

  Usuwa element z początku.
- void **remove_at** (int index)

  Usuwa element z końca.
- void clear ()

  Usuwa element z podanego indeksu.
- int **getSize** () const

  Czyści całą listę
- ListIterator **begin** ()
- ListIterator **end** ()

  Daje iterator na początek listy.
- ListIterator **rbegin** ()

  Daje iterator na koniec (poza ostatni element).
- ListIterator **rend** ()

  Daje iterator na początek listy od końca.

3.1.1 Dokumentacja konstruktora i destruktora

3.1.1.1 ~DoublyLinkedList()

```
DoublyLinkedList::~DoublyLinkedList ()
```

Pętla zwalniająca pamięć z RAMu, żeby nie było wycieków

3.1.2 Dokumentacja funkcji składowych

3.1.2.1 clear()

```
void DoublyLinkedList::clear ()
```

Usuwa element z podanego indeksu.

Resetujemy wskaźniki po wyczyszczeniu listy

Resetujemy rozmiar listy

3.1.2.2 pop_back()

```
void DoublyLinkedList::pop_back ()
```

Usuwa element z początku.

Lista stała się pusta

3.1.2.3 pop_front()

```
void DoublyLinkedList::pop_front ()
```

Dodaje element z podanego indeksu.

Lista stała się pusta

3.1.2.4 push_back()

```
void DoublyLinkedList::push_back (
            int val)
```

Dodaje element na początek.

Używamy fabryki do stworzenia węzła!

Jeśli lista była pusta, nowy element jest i głową i ogonem

Stary ogon wskazuje na nowy element

Nowy element wskazuje wstecz na stary ogon

Nowy element staje się oficjalnym ogonem

3.1.2.5 push_front()

```
void DoublyLinkedList::push_front (
            int val)
```

Destruktor - czyści pamięć po usunięciu listy.

Używamy fabryki do stworzenia węzła!

Jeśli lista była pusta, nowy element jest i głową i ogonem

Nowy element wskazuje na stary początek

Stary początek wskazuje wstecz na nowy element

Nowy element staje się oficjalną głową

Dokumentacja dla tej klasy została wygenerowana z plików:

- DoublyLinkedList.h
- DoublyLinkedList.cpp

3.2 Dokumentacja klasy ListIterator

Metody publiczne

- **ListIterator** (Node ∗ptr)

  Pamięta, na którym elemencie aktualnie stoimy.
- int & **operator**∗ ()
- ListIterator & operator-- ()

  Pozwala odczytać wartość elementu, na którym stoimy (gwiazdka ∗it).
- ListIterator & operator++ ()

  Pozwala przejść do poprzedniego elementu (–it).
- bool operator== (const ListIterator &other) const

  Pozwala przejść do następnego elementu (++it).
- bool operator!= (const ListIterator &other) const

  Sprawdza, czy iteratory wskazują na ten sam element.
- Node ∗ **getNode** () const

  Sprawdza, czy to już koniec listy.

3.2.1 Dokumentacja funkcji składowych

3.2.1.1 operator"!=()

```
bool ListIterator::operator!= (
            const ListIterator & other) const
```

Sprawdza, czy iteratory wskazują na ten sam element.

Porównuje, czy dwa iteratory wskazują na to samo miejsce

3.2.1.2 operator++()

```
ListIterator & ListIterator::operator++ ()
```

Pozwala przejść do poprzedniego elementu (–it).

Zwraca dane z aktualnego węzła. Przesuwa wskaźnik na kolejny element w prawo

3.2.1.3 operator--()

```
ListIterator & ListIterator::operator-- ()
```

Pozwala odczytać wartość elementu, na którym stoimy (gwiazdka ∗it).

Pozwala przejść do poprzedniego elementu lub do nullptr (co kończy pętlę rend)

3.2.1.4 operator==()

```
bool ListIterator::operator== (
            const ListIterator & other) const
```

Pozwala przejść do następnego elementu (++it).

Porównuje, czy dwa iteratory wskazują na ten sam element

Dokumentacja dla tej klasy została wygenerowana z plików:

- Iterator.h
- Iterator.cpp

3.3 Dokumentacja klasy Node

Klasa przechowująca pojedynczy element listy dwukierunkowej.

```
#include <node.h>
```

Metody publiczne

- Node (int val)

  Wskaźnik na poprzedni element.

Atrybuty publiczne

- int **data**
- Node ∗ **next**

  Tutaj trzymamy wartość
- Node ∗ **prev**

  Wskaźnik na następny element.

3.3.1 Opis szczegółowy

Klasa przechowująca pojedynczy element listy dwukierunkowej.

Klasa odpowiada za trzymanie danych całkowitych oraz wskaźników na poprzedni i następny węzeł.

3.3.2 Dokumentacja konstruktora i destruktora

3.3.2.1 Node()

```
Node::Node (
            int val)
```

Wskaźnik na poprzedni element.

Konstruktor klasy Node.

Parametry

val	Wartość typu int, jaka ma być przypisana do węzła.

Przypisujemy podaną wartość

Na start nie ma następnika

Na start nie ma poprzednika

Dokumentacja dla tej klasy została wygenerowana z plików:

- node.h
- node.cpp

3.4 Dokumentacja klasy NodeFactory

Fabryka tworząca nowe obiekty węzłów listy.

```
#include <NodeFactory.h>
```

Statyczne metody publiczne

- static Node * createNode (int val)
  Tworzy nowy węzeł listy na stercie.

3.4.1 Opis szczegółowy

Fabryka tworząca nowe obiekty węzłów listy.

Klasa implementuje wzorzec fabryki w celu odseparowania logiki alokacji pamięci na stercie od zarządzania samą strukturą listy.

3.4.2 Dokumentacja funkcji składowych

3.4.2.1 createNode()

```
Node * NodeFactory::createNode (
            int val)  [static]
```

Tworzy nowy węzeł listy na stercie.

Parametry

val	Wartość typu int przekazywana do nowego węzła.

Zwraca

Wskaźnik na nowo utworzony obiekt typu Node.

Fabryka tworzy w pamięci komputera nowy węzeł i go zwraca

Dokumentacja dla tej klasy została wygenerowana z plików:

- NodeFactory.h
- NodeFactory.cpp

Rozdział 4

Dokumentacja plików

4.1 DoublyLinkedList.h

```
00001 #ifndef DOUBLY_LINKED_LIST_H
00002 #define DOUBLY_LINKED_LIST_H
00003 #include "Node.h"
00004 #include "Iterator.h"
00005
00006 class DoublyLinkedList {
00007 private:
00008     Node* head;
00009     Node* tail;
00010     int size;
00011 public:
00012     DoublyLinkedList();
00013     ~DoublyLinkedList();
00014
00015
00016     void push_front(int val);
00017     void push_back(int val);
00018     void insert_at(int index, int val);
00019
00020
00021     void pop_front();
00022     void pop_back();
00023     void remove_at(int index);
00024
00025     void clear();
00026
00027     int getSize() const;
00028     ListIterator begin();
00029     ListIterator end();
00030     ListIterator rbegin();
00031     ListIterator rend();
00032 };
00033 #endif
```

4.2 Iterator.h

```
00001 #ifndef ITERATOR_H
00002 #define ITERATOR_H
00003 #include "Node.h"
00004
00005 class ListIterator {
00006 private:
00007     Node* current;
00008 public:
00009     ListIterator(Node* ptr);
00010     int& operator*();
00011     ListIterator& operator--();
00012     ListIterator& operator++();
00013     bool operator==(const ListIterator& other) const;
00014     bool operator!=(const ListIterator& other) const;
00015     Node* getNode() const;
00016 };
00017 #endif
```

4.3 Dokumentacja pliku node.h

Definicja klasy Node reprezentującej pojedynczy węzeł listy.

Komponenty

- class Node

  Klasa przechowująca pojedynczy element listy dwukierunkowej.

4.3.1 Opis szczegółowy

Definicja klasy Node reprezentującej pojedynczy węzeł listy.

4.4 node.h

Idź do dokumentacji tego pliku.

```
00001 #ifndef Node_h // Zabezpieczenie przed wielokrotnym wczytaniem pliku
00002 #define Node_h
00003
00008
00016 class Node {
00017 public:
00018     int data;
00019     Node* next;
00020     Node* prev;
00021
00026
00027
00028     Node(int val);
00029 };
00030 #endif
```

4.5 Dokumentacja pliku NodeFactory.h

Definicja klasy NodeFactory odpowiedzialnej za wzorzec fabryki węzłów.

```
#include "node.h"
```

Komponenty

- class NodeFactory

  Fabryka tworząca nowe obiekty węzłów listy.

4.5.1 Opis szczegółowy

Definicja klasy NodeFactory odpowiedzialnej za wzorzec fabryki węzłów.

4.6 NodeFactory.h

Idź do dokumentacji tego pliku.

```
00001 #ifndef NodeFactory_h
00002 #define  NodeFactory_h
00003 #include "node.h"
00008
00009
00010
00018
00019 class NodeFactory {
00020 public:
00021
00027     static Node* createNode(int val);
00028
00029 };
00030 #endif
00031
```